Exercise - RTL, understanding/extending Croc

This exercise is still being updated for the 2026FS class. You can look at it but don't expect it to be in its final version until this message is displayed.

What you will do

In this exercise, you will dive into the hardware description of an example of System-on-Chip (SoC) written in System Verilog. You may be familiar with certain computer architecture principles such as buses, memory maps, and memory-mapped inputs/outputs (MMIOs). Today you will see how these architectural elements are implemented using hardware description languages (HDLs). You will browse the codebase of an open-source RISC-V based 32-bit SoC: Croc. You will identify the following elements:

- Memory map and interconnects: you will retrieve the memory map of Croc from its hardware description.
- On-chip communication protocols and peripherals: you will extend a simple read-only memory (ROM) using the Open Bus Interface (OBI) protocol used in Croc.
- MMIOs: you will modify the software application running on Croc's CVE2 core to interact with devices.

Finally, you will look at a more complex peripheral using combinatorial and sequential logic to refresh System Verilog usage. This will be a memory-mapped IO module that accelerates a very useful yet costly operation: computing the number of set-bits in an array of 32-bit words.

About the style

We will use a number of different styles to identify different types of actions as shown below:

Student Task

Parts of the text that have a gray background, like the current paragraph, indicate steps required to complete the exercise.

Actions that require you to select a specific menu will be shown like the following: `menu → sub-menu → sub-sub-menu`

Whenever there is an option or a tab that can be found in the current view/menu we will use a `BUTTON` to indicate such an option.

Throughout the exercise you will be asked to enter certain commands using the command-line.

```
sh> command to be entered on the Linux command line
```

Note: *there are many reasons for using a command-line. Some functionality can not not be accessed through GUI commands, and in some cases, using the command-line will be much faster. Most importantly, things you enter on the command-line can be converted into a script and executed repeatedly.*

Preparation

Student Task 1

For ETH users:

- Setup the working directory for this exercise by calling our install script and navigating to the newly created directory. **Important**: make sure you install your exercises on the local hard disks rather than in the accounts home directories (which have a limited disk quota). For this sake, create a directory with your username on the local hard disk (e.g. `/scratch/vlsi2_XXfs26/`), and install the exercise files there. Check out the dedicated Exercise setup page for more information.

```
sh > mkdir -p /scratch/$USER
sh > cd /scratch/$USER
sh > vlsi2 ex03
sh > cd ex03
```

Alternatively, you can directly call the install script for each exercise which you can find under `/home/vlsi2/ex03/install.sh`.

For non-ETH users:

Note: The exercises have been recently updated for the Spring semester 2026. It is possible that some exercise branches on Github require some additional time to be updated accordingly. Please reach out to a maintainer in case of issues.

- Retrieve the files from this exercise from Github:

```
sh > git clone https://github.com/pulp-platform/croc.git
sh > cd croc
sh > git checkout ex03
```

Exploring the Croc system-on-chip

See below the architecture of Croc (details are available in the dedicated page). The main processor core is CVE2: a fork of the RISC-V 32 Ibex core. This core is connected to the rest of the system (memory and peripherals) via a system bus using the OBI protocol.

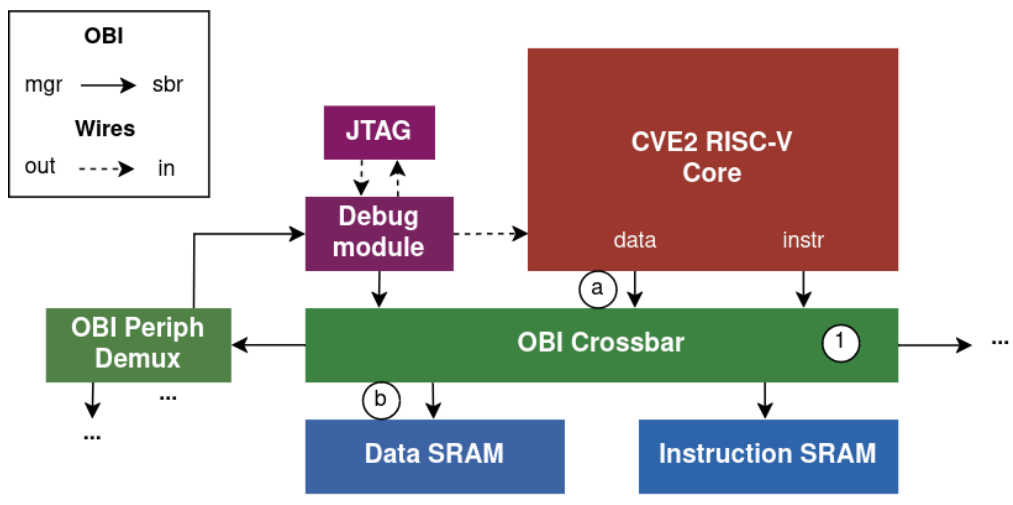

Figure 1: Simplified Croc diagram

System Bus and OBI Protocol

The system bus is the backbone of a system-on-chip: it connects the processor core to the memory. In Croc, this bus is implemented in hardware using a crossbar. This hardware block links each bus manager (mng) to each bus subordinate (sbr) in an all-to-all fashion. In Figure 1, the interfaces between elements are represented by arrows, starting from the manager to the subordinate. For instance, the CVE2 core is connected to the Data SRAM via two interfaces:

- Ⓐ CVE2 data port (mgr) - OBI crossbar port (sbr)
- Ⓑ crossbar port (mgr) - Data SRAM (sbr)

Communication between modules may require multiple pieces of information (data, addresses, read/write selection, ...). To simplify the overall architecture, buses implement standardized protocols. In Croc, we use the OBI protocol.

Student Task 1

To understand the OBI protocol, we will look at example communication traces within Croc. First, read the OBI page to learn how the protocol works better.

- Open the trace file *task1.vcd* (in the snippet below, the current directory is in parenthesis, "croc" here).

First, open a bash session with `oseda`. This will add the required programs in your `$PATH`.

```
sh (ex03) > oseda -2025.12 bash
```

Then, navigate to the `verilator` directory and open the provided waveform with GTKWave:

```
sh (ex03)      > cd verilator/
sh (verilator) > gtkwave task1.vcd
```

This trace contains some OBI transactions between the processor core and an endpoint. We prepared a GTKWave file to display these.

- Load the GTKWave file: select `File > Read Save File` and load *task1.gtkw*.

For now, let's focus on the transactions received **at the endpoint** (see the signals below `Endpoint A` and `Endpoint R`).

- How many transactions did the core send to the endpoint, and how did you count them? Remember that a transaction contains both a request (A) and a response (R).

Please write down your answer - and all answers to future questions - on paper to discuss with your teaching assistants.

- **For all transactions at the endpoint** write down the transaction's address, data, id and type (write/read).
- The last transaction is corrupted and may cause the system to hang. Can you identify the issue?

ezelioli -- We can switch to the Surfer waveform viewer (optional)

Note: You may have noticed that during some transactions, the `wdata[31:0]` field is not null. The value of this field is not specified by the protocol when `we` is low. Thus, the manager does not need to ensure that `wdata[31:0]` is kept low during a read transaction.

System bus and Memory Map

You have now observed a few OBI transactions at one of the subordinate endpoints. However, Figure 1 shows that the processor core can access multiple peripherals and memories. The role of the system bus is not only to wire elements between them but also to appropriately route requests between managers and subordinates.

The routing happens in the crossbar ①: each transaction is forwarded to a subordinate port based on its address field. To display the routing tables from the processor core to all the other elements in the system, we build what we call an **address map**.

Region			Start	End
System Bus	Peripheral Bus	Debug	?	?
		?	?	?
		?	?	?
		?	?	?
		?	?	?
		?	?	?
	SRAM 0		?	?
	?		?	?
	User Bus	?	?	?
		?	?	?

Memory map of Croc.

This address map is then implemented in hardware using modules such as **crossbars** and **demultiplexers**. While the former connects managers to subordinates in an **all-to-all** fashion, the latter connects one manager to subordinates in a **one-to-all** fashion. You can see in Figure 1 the *OBI Crossbar* and *OBI Periph Demux*; note that some elements are intentionally missing as you will need to complete this diagram in the next task.

Student Task 2

The address map for this chip is divided into three main address regions, but in Figure 1 you can see that one region (i.e. bus) is missing.

- Reproduce Figure 1 on paper and add the third bus for the User address region; it is implemented in hardware using a demultiplexer.

We will now browse the System Verilog code to finish populating your schematics and the address map.

- Open *croc_pkg.sv*, *croc_domain.sv*, *user_domain.sv* and *user_pkg.sv* (for now ignore the `// TODOs` until it is explicitly asked to implement them). The `*_pkg.sv` files contain shared definitions such as parameters and types. The `*_domain.sv` files instantiate and connect the modules.

If you can't find a file, you can use the following command

```
sh (ex03) > find . -name "croc_pkg.sv"
```

If you are using VSCode, you can also use `Ctrl+P` to find a file.

The three buses are implemented by one `obi_xbar` and two `obi_demux`.

- Find the three bus blocks and annotate your diagram with the file name where they are instantiated.

Let's focus on the system bus first. Its address map is defined in `croc_addr_map` (available in the `croc_pkg`).

- Fill in the address map with the system bus information.
- Now, fill in the address map the other two buses. Add also the associated peripherals to your schematic.

Note: *You may now discuss your answers with a teaching assistant.*

Now that you have drawn the system map, we will give a new look to the waveform provided.

Student Task 3

We previously looked at the peripheral transactions, and we will now look at the transactions **coming out of CVE2**.

- Re-open the wave file provided.
- How many transactions can you count **at the CVE2 data port**?
- Is this number higher that the one you reported at the endpoint? Why?
- For each transaction, write down the subordinate and the address offset of the transaction within the endpoint.

Memory-Mapped IO (MMIO)

Now that you have a complete view of Croc, we will see how CVE2 interacts with the different subordinates.

These modules are referred to as memory-mapped peripherals. If the processor wants to control them, it does not need to use specific instructions but simply memory transactions (loads and stores) to their associated addresses. For instance, to write the character "H" on the UART port, the core will write the ASCII value "72" at the UART address.

```c
#include <inttypes.h>

void uart_write(uint8_t byte) {
    // Declare the uart region starting at 0x03002000
    volatile void *uart_base = (volatile void *) 0x03002000;
    // Write a uint8_t at offset 0
    *(volatile uint8_t *)(uart_base + 0x0) = byte;
}

int main() {
    uart_write('H');
    uart_write('i');
    uart_write('!');
    uart_write('\n');
}
```

The code above shows a simple program that prints a few characters to UART. Note that we use `void *` to perform pointer arithmetic because this type is equivalent to an integer. But before using the pointer, we need to cast it to the data size we want to use (here: `uint8_t *`).

Student Task 4

- Open the compiled version of this program in Godbolt (https://godbolt.org/z/vs8eGaP6P). Which RISC-V instructions will trigger the

UART transmission?

Note: The actual implementation of `uart_write` is available in *uart.c*.

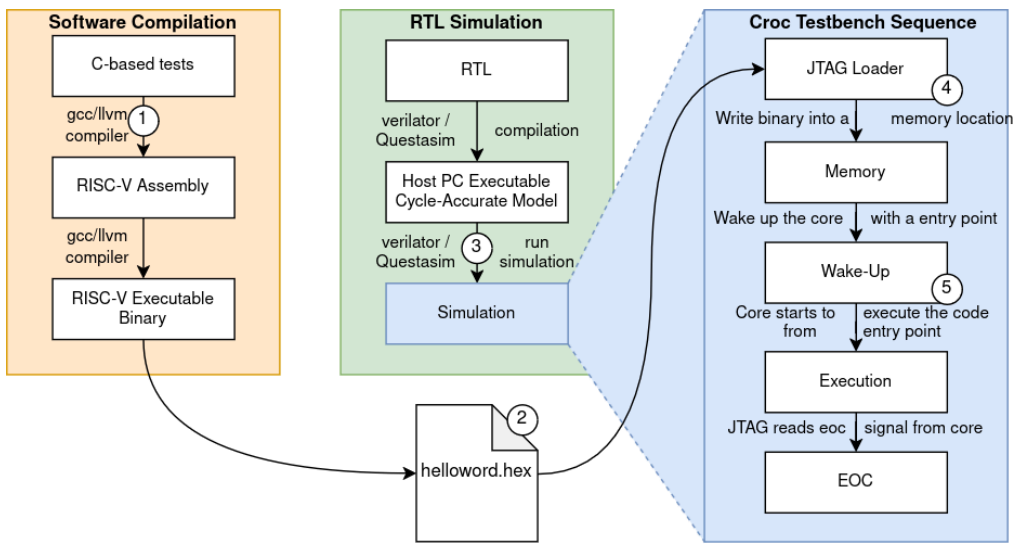

Figure 2: The Croc test simulation flow.

Figure 2 shows the RTL test flow for Croc. A "test" is an application that gets executed on the CVE2 core ①. This application can access different SoC resources to provide stimuli for hardware modules. This test is compiled from C sources to a binary file for RISC-V 32-bit ②. The RTL code of Croc and its testbench are compiled into a cycle-accurate model ready for simulation ③. At the beginning of the simulation, the system testbench reads the test binary and loads it into the SoC memory ④. Once the memory is loaded, the testbench can wake up the core by de-asserting resets, and the code will start executing instructions at a given location (where the test program was previously loaded) ⑤.

Student Task 5

You will now write the code to access the user Read-Only Memory (ROM).

- Open *helloworld.c*, and implement `// TODO :` read eight 32-bit words (from offset `0x0` to `0x20`) from the ROM and print them. Use the function `reg32` to read a 32-bit word. (The function `reg32` is defined in *sw/lib/inc/util.h*) You can then print this word as hexadecimal using `printf` and `uart_write_flush`. Attention, the simple implementation of `printf` only supports `"%x %c"`.
- Compile the elf file.

```
sh (sw) > make bin/helloworld.elf
```

- Observe the make output. What compiler did it use, and for what microarchitecture? Can you execute this code on your machine?
- You will now extract the data and instructions from this elf file. These will be later loaded into the SRAMs within the Croc simulation. To read an elf file, you can use the GNU tool `objcopy` .

```
sh (sw) > riscv64-unknown-elf-objcopy -O verilog bin/helloworld.elf bin/helloworld.hex
```

- Finally, if you want to see the RISC-V instruction executed during the simulation, you can use `objdump` .

```
sh (sw) > riscv64-unknown-elf-objdump -D -s bin/helloworld.elf >bin/helloworld.dump
```

Note: In the commands above, `riscv64-unknown-elf` is called the "triple"; this identifies the toolchain that you are using. However, the same toolchain can support different flavors and extensions of the same ISA. This is why we add the *-march* argument to specify the exact architecture we are targeting. We do not do this for `objcopy` and `objdump` as *rv32i* and *rv64i* opcodes overlap.

The user ROM is implemented in System Verilog in the file *user_rom.sv*. It implements the following requirements:

1. When a request is received, it must be granted within the same cycle.
2. When a request is granted, the response must be valid on the next cycle.
3. If the request is a write, the response must be an error.
4. The response id must be the same as the request id.
5. The ROM capacity is four 32-bit words.

6. Un-aligned accesses are not supported, i.e., read at addresses 0x0, 0x1, 0x2, and 0x3 will all return the content of the first 32-bit (4 bytes) word.
7. Byte granularity accesses are ignored, i.e., the read response value will be the same regardless of the byte-enabled field.

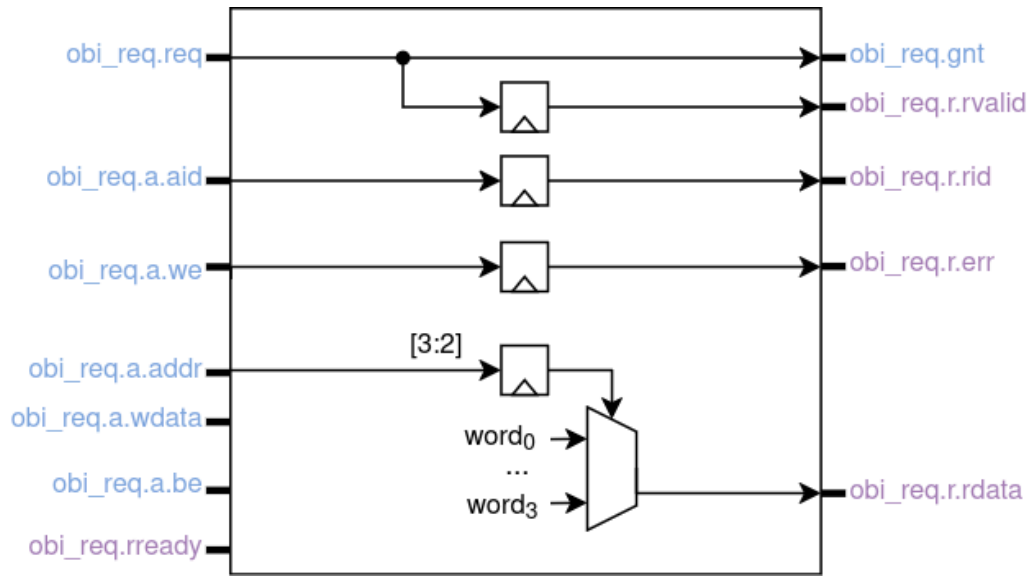

Figure 3: Simple logic between some of the ROM outputs and inputs.

Now that you have built your test program (orange box in Figure 2), you will compile your SoC with the simulator (green box) and execute the test program in the simulated core (blue box).

Student Task 6

- First, compile first the SoC testbench (if you opened a new terminal, always remember to start the container shell with `oseda bash`). To do so, navigate to the `verilator` directory and execute the following command (this should take approximately one minute).

```
sh (ex03)      > cd verilator
sh (verilator) > verilator --binary -j 0 -Wno-fatal -Wno-style --timing --autoflush --trace-fst --trace-structs --trace-threads 2 --unroll-count 1 --unroll-stmts 1 --x-assign fast --x-initial fast -O3 --top tb_croc_soc -f croc.f
```

In this way, the SoC will be compiled into *croc.fst*. The FST format is smaller and opens faster than the VCD format we used before. You can also switch the flag --trace-fst to --trace if you prefer the VCD format.

We also prepare you a verilator script called *run_verilator.sh*, so that instead of typing the tedious extremely long command as above, you can compile the simulation model more conveniently by:

```
sh (verilator) > ./run_verilator.sh --build
```

You can see an overview of the script's functionality with:

```
sh (verilator) > ./run_verilator.sh --help
```

- Now, run the simulation with the *helloworld.hex* binary and check for its completion: "Simulation finished: return code (...)"

```
sh (verilator) > ./obj_dir/Vtb_croc_soc +binary="../sw/bin/helloworld.hex"
```

This execution can also be run with the *run_verilator.sh* script:

```
sh (verilator) > ./run_verilator.sh --run ../sw/bin/helloworld.hex
```

Note: In case of hardware error, the core will automatically reboot. If you see "*hello world*" printing more than once, you can interrupt the simulation and go to the next step.

- Open the waveform for this simulation:

```
sh (verilator) > gtkwave croc.fst &
```

On the left panel, under *TOP*, you will find the simulation testbench `tb_croc_soc`, which contains the design under test: `i_croc_soc`. Directly under *TOP*, you will also find the different SystemVerilog packages used in this design.

- Recursively import the signals in `tb_croc_soc/i_croc_soc/i_user/i_user_rom`, what values are read at address 0x0, 0x4, ..., 0x1c - and printed to the simulation console?

You can also save this waveform configuration file by selecting `File > Write Save File As` *croc.gtkw*, so that you can quickly import the signals again by selecting `File > Read Save File` and loading *croc.gtkw*, instead of finding the signals recursively every time you open the simulation model.

You will now modify the behavior of our User ROM. Let's assume we want it to assign the response two cycles after the request instead of one.

Student Task 7

- Open *user_rom.sv*. Again, you can find this file with the `find` command or `Ctrl+P` in VScode.
- Implement `// TODO 1 :` modify the code such that the User ROM will respond after two cycles instead of one. If you implement a flip-flop, make sure that you use the reference code in the [EDA Wiki (http://eda.ee.ethz.ch/index.php?title=SystemVerilog_Examples#Registers)](http://eda.ee.ethz.ch/index.php?title=SystemVerilog_Examples#Registers). If you want to know the content of the `obi_req_t` and `obi_rsp_t` structs, you can find their declaration in *croc_pkg.sv*.
- Recompile your module and re-run the simulation. You can reload the wave visualization by clicking ↻ in the GTKWave GUI toolbar.

Compare your waveform with the one in Figure 4 below. Note that depending on your C code the core may be slower to issue ROM read requests.

Note: if the compilation fails, you can redirect *stdout* and *stderr* to a file:

```
sh (verilator) > verilator <options..> 2>&1 | tee compile.log
```

The redirection has also been done in the *run_verilator.sh* script, so every time you run `./run_verilator.sh --build`, there is a *croc_build.log* automatically saved in this verilator directory.

- In your simulation, is the CVE2 core slower to issue requests than in Figure 4? If yes, do you have an idea why? You don't need to implement changes now.

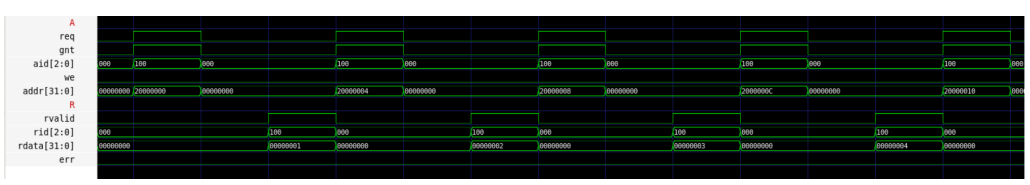

Figure 4: Expected waveform for OBI requests and responses within the modified User ROM module.

Note: *You may now discuss your answers with a teaching assistant.*

Student Task 8

Let's now change and personalize the content of your User ROM.

- You can see that the original ROM can hold only four ASCII characters at most. Modify the ROM to be able to hold 32 ASCII characters.
- Sign your future chip! Use `hexdump` and fill your ROM with your initials (make sure that it ends with the ASCII character `'\0'`.

You can know the hexadecimal value of your initials using this shell command:

```
sh > echo "[your initials]'s ASIC" | hexdump
```

- Compile and simulate the Croc testbench with your modified User ROM using Verilator. Keep your simple test program of before; can you see the first hexadecimal word?
- Change *helloworld.c* to print a string in ASCII characters instead (`"%s"`). Note that you can compile the HEX file by directly using this command:

```
sh (sw) > make bin/helloworld.hex
```

- What is the endianness of CVE2? Modify your User ROM so the print shows correctly.

You have now seen how to write C code to access MMIO, how to explore the Croc RTL, and how to modify a simple peripheral. In the next tasks, you will work with a more difficult peripheral that can accelerate algorithms.

A more complex peripheral: a popcount accumulator

Counting the number of bit sets in a register is an essential operation in many error detection and compression algorithms. For this reason, many processors support this operation directly in hardware. For instance, in the base ARM ISA, with the `CNT` instruction. Unfortunately, the base RISC-V ISA does not contain such instruction, thus requiring software implementation instead:

```
unsigned int count_set_bits(unsigned int n)
{
    unsigned int count = 0;
    while (n) {
        count += n & 1;
        n = n >> 1;
    }
    return count;
}

int main() {
    // Count the number of bits in an array
    int arr[10];
    unsigned int acc = 0;
    for(int i = 0; i < 10; i++)
        acc += count_set_bits(arr[i]);
}
```

Compiling `count_set_bits` with *-O3* optimizations returns the following assembly:

```
count_set_bits(unsigned int):
        mv      a5,a0
        li      a0,0
        beq     a5,zero,.L4
.L3:
        andi    a4,a5,1
        srli    a5,a5,1
        add     a0,a0,a4
        bne     a5,zero,.L3
        ret
.L4:
        ret
```

On our 32-bit machine, the loop `.L3` might take up to 128 instructions to execute if the most significant bit is set in the input variable `n`. To address this inefficiency, we prepared a dedicated hardware module to accelerate this operation. In hardware design and computer architecture, the operation of counting the number of set bits is standardly referred to as population count, or popcount for short. Therefore, we have named our accelerator *user_popcount_acc* (User Population Count Accumulator).

We will describe later its detailed functionality. For now, we will connect this new peripheral to the user bus.

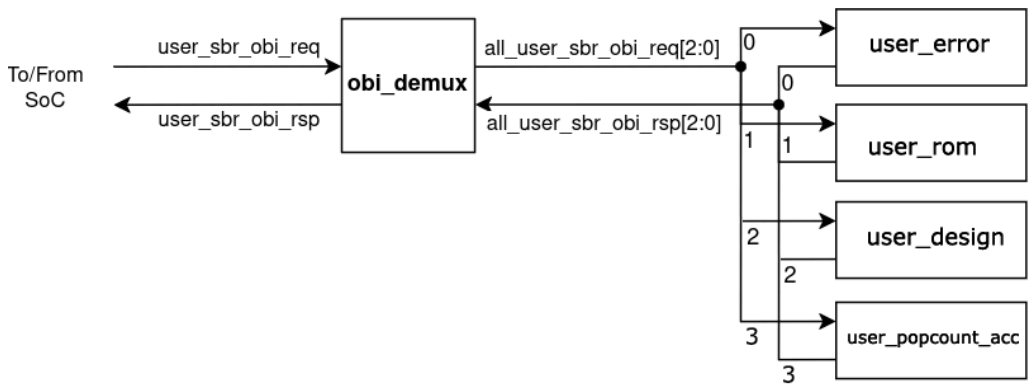

Figure 5: Connections from the user OBI demultiplexer.

In *user_domain.sv* and *user_pkg.sv*, the user bus is represented in HDL by the `i_user_demux` instance and the associated signals `all_user_sbr_obi_req` and `all_user_sbr_obi_rsp`. These signals connect the demultiplexer and the user peripherals (OBI subordinates). `all_user_sbr_obi_req` is defined as an array of size `NumDemuxSbr` containing elements of type `sbr_obi_req_t`.

Student Task 9

Lets connect `user_popcount_acc` at address `0x20002000` and allocate a 4KiB region to it (you can use [this website (https://ss64.com/convert.html)](https://ss64.com/convert.html) to convert KiB to hexadecimal).

- Open *user_domain.sv*, *user_pkg.sv* and *user_popcount_acc.sv*

Let's start by implementing the `// TODOs` in *user_domain.sv* and *user_pkg.sv*.

- Implement `// TODO 1 :` (*usr_domain.sv*) declare the signals connecting the new module and the OBI user demultiplexer.
- Implement `// TODO 2 :` (*usr_pkg.sv*) a one-to-all bus can be represented as an array of requests and responses (see Figure 5), declare the array index for the new module.
- Implement `// TODO 3 :` (*usr_domain.sv*) connect the signals from `// TODO 1` to the demultiplexer using the arrays `all_user_sbr_obi_req` and `all_user_sbr_obi_rsp`.
- Implement `// TODO 4 :` (*usr_domain.sv*) instantiate and connect `user_popcount_acc`.
- Make sure that your RTL compiles and simulates properly, and use GTKWave to browse your Design Under Test (DUT) and verify that you find the signals and instances you added.

Now, the new module is part of the SoC, and connected to the OBI user bus at address `0x20002000`. Let's verify its functionality.

user_popcount_acc has the following memory-mapped register interface:

Offset	Name	Access	Description
0x0	reset_accumulator	Hardware: Read Software: Write	When read, this returns an error. When written, the accumulator is set to zero.
0x4	push_value	Hardware: Read Software: Write	When written, the number of set bits in the request's write data is added to the accumulator. Reading returns an error.
0x8	accumulator	Hardware: Read/Write Software: Read	When read, this returns the accumulator. Writing returns an error.

Student Task 10

Let's execute the test that we prepared to verify the functionality of this block.

- Let's compile the test and execute it. Again, remember that you need to start a new bash with `oseda` if you closed your terminal.

```
sh (sw) > make bin/test_popcountacc.hex
sh (verilator) > ./run_verilator.sh --build --run ../sw/bin/test_popcountacc.hex
```

- The result is printed at the end of the simulation, is it correct?
- Open *user_popcount_acc.sv*.
- Implement `// TODO 1:` with the functionality of the `reset_accumulator` register.
- Implement `// TODO 2:` with the logic to count the number of set-bits.
- Verify using the software test. What speedup do you get between the software and hardware implementation?

You are done with the exercise. Congratulations!

Discuss your experience with assistants and colleagues, or continue to explore Croc and think about cool accelerators/peripherals.

Student Task Bonus Task 1

The module `user_popcount_acc` have significantly improved the runtime required to count the set-bits in an array. However, the C code provided in *test_popcountacc.c* is not the most optimized one. Using `GCC` or C code, try to improve the runtime for the hardware implementation.

The VLSI pages are part of the open source VLSI design course offered by the Integrated Systems Laboratory (https://iis.ee.ethz.ch) of ETH Zürich (https://www.ethz.ch), by Luca Benini (https://ee.ethz.ch/the-department/people-a-z/person-detail.luca-benini.html) and Frank K. Gürkaynak (https://iis-people.ee.ethz.ch/~kgf/). See full list of contributors. Unless otherwise noted content available under Creative Commons BY (https://creativecommons.org/licenses/by/4.0/).